

More Strings

CSCI 1101

2024-02-02

String Length

A **string** is a sequence of **characters**.

To find out how many we use the length function,

```
len : (str) --> int
```

```
len("Bowdoin")
```

```
len(str(2 + 2))
```

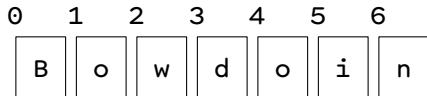
```
len("")
```

In python the character type is not separate from the string type.
Instead, characters are just strings of length 1.

String Indexing

To get the character at a given **offset** from the beginning of a string we use the **index operator**, `_[_]` : `(str , int) --> str`

The character “at” a given index is the one that “starts” at that offset.



`"Bowdoin"[0]`

`"Bowdoin"[1]`

`...`

`"Bowdoin"[5]`

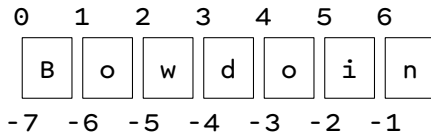
`"Bowdoin"[6]`

It is an error to index out of bounds :

`"Bowdoin"[7]`

Negative Indexing

For convenience, you can use a negative index to “count backwards” from the end of a string.



`s[-n]` is just shorthand for `s[len(s) - n]`.

`"Bowdoïn"[-1]`

`"Bowdoïn"[-2]`

`...`

`"Bowdoïn"[-6]`

`"Bowdoïn"[-7]`

String Slicing

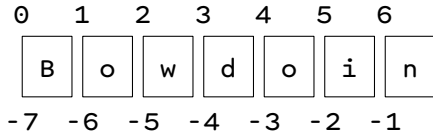
A sequence of consecutive characters in a string is called a **substring**.

For example, "ell" is a substring of "hello".

To get the substring between a given pair of offsets in a string we use the **slice operator**, `str[start:end]` : `(str , int , int) --> str`.

- The **start index** is the offset of the first character that you want to *include*.
- The **stop index** is the offset of the first character that you want to *exclude*.
- If the stop index is at, or to the left of, the start index then the slice is empty.
- If the start index is omitted then `0` is implicitly assumed.
- If the stop index is omitted then `len(s)` is implicitly assumed.
- It is not an error to index out of bounds (but there's nothing there).

String Slicing Practice



Try to predict the values of the following string slice expressions:

`"Bowdoin"[0 : 3]`

`"Bowdoin"[3 : 7]`

`"Bowdoin"[1 : 5]`

`"Bowdoin"[3 : 3]`

`"Bowdoin"[4 : 3]`

`"Bowdoin"[: -3]`

`"Bowdoin"[-2 : ]`

`"Bowdoin"[-100 : 100]`

Activity: Sting Slicing

Practice string slicing by writing a function

```
rotate : (int , str) --> str
```

that breaks the argument string into two pieces at the specified index and concatenates those pieces in the opposite order.

This has the effect of moving the specified number of characters from the start of the string to the end.

Your function should behave like this:

```
rotate(3 , "Bowdoin")  
'doinBow'
```

String Functions

Python comes with some built-in functions for working with strings.

Many of them are organized in the `str` module.

For example:

```
str.upper    :   (str) --> str  
str.lower    :   (str) --> str  
str.title    :   (str) --> str
```

Try calling these on the string "iT wAs ThE bEsT oF tImEs" to see what they do.

String Functions ctd.

To remove extraneous leading and trailing whitespace from text use

```
str.strip : (str) --> str
```

For example:

```
str.strip(' it was the best of times ')
```

To replace all non-overlapping occurrences of a substring in a string use

```
str.replace : (str , str , str) --> str
```

For example:

```
str.replace(' it was the best of times ' , ' ' , '_')
```

Activity: Favorite Books

Write a function `fav_book : () --> NoneType` that asks the user for their favorite book, and declares that that's its favorite book too.

Your function should quote the book title, remove any leading and trailing spaces, and capitalize each title word, like this:

```
What is your favorite book?    the grapes of WRATH
What a coincidence, "The Grapes Of Wrath" is my favorite
    book too!
```

To Do

Finish *Homework 2: Strings* on CodeRunner by 11:00AM next Thursday.